

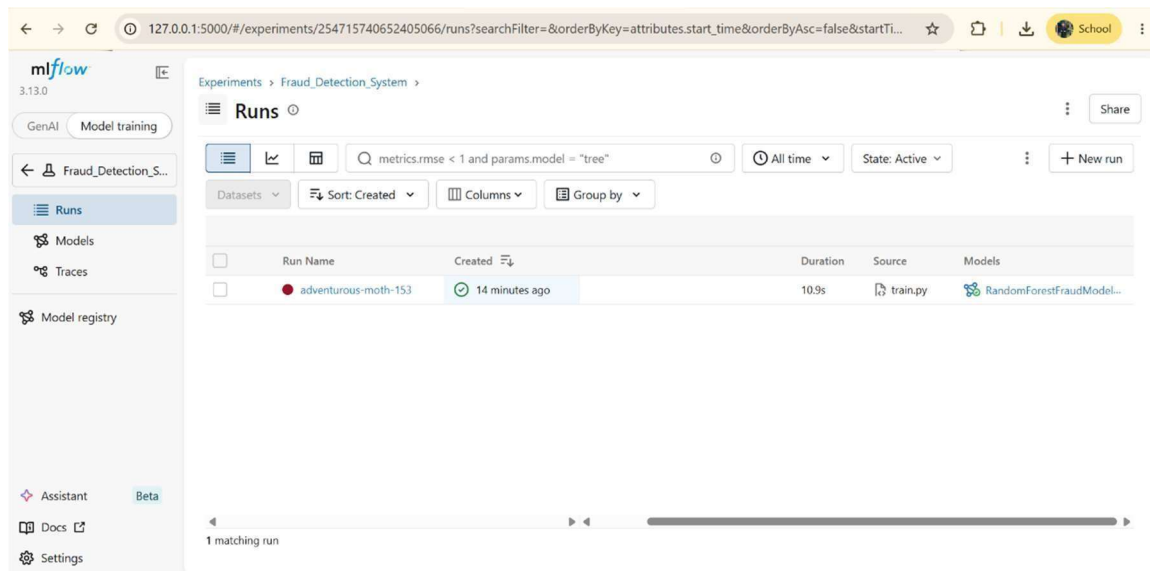
THE UNIVERSITY OF LAHORE

Department of Intelligent Systems - Spring Semester 2026

Name:	Ahad Ali	Subject:	MLOPS
Section:	BSES	Project # :	4
Sap ID:	70150510	Submitted To:	Sir Sikander Hayat

Model Training & Experiment Tracking (MLflow) :

MLflow Experiment Dashboard & Run Logs:



The initial phase of the pipeline focuses on experiment tracking and model lineage using MLflow. As shown in the dashboard, a training run named 'adventurousmoth-153' was logged successfully via train.py.

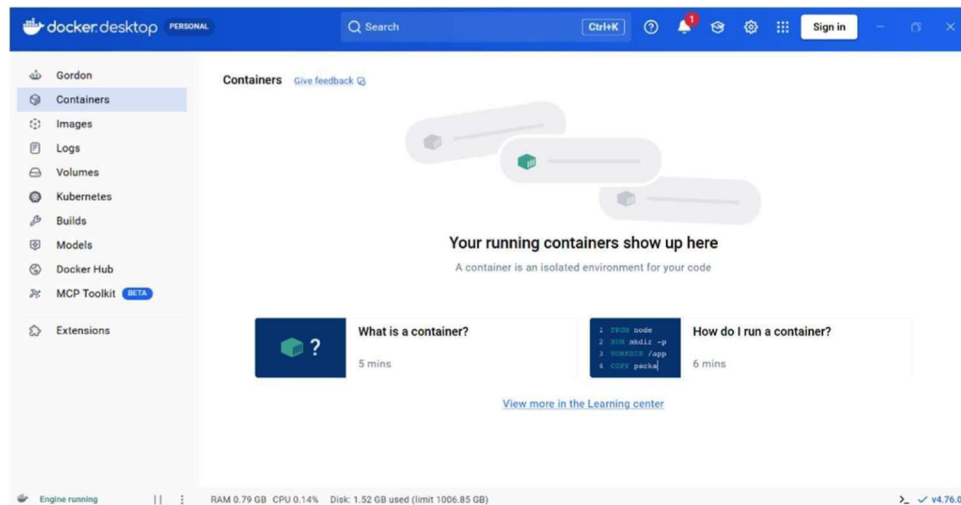
Production Workspace & Environment Setup :

Clean Production Directory Structure:

Name	Date modified	Type	Size
tracking_data	6/4/2026 12:19 AM	File folder	
deploy	6/4/2026 4:39 PM	Windows PowerShell ...	2 KB
Dockerfile	6/4/2026 4:30 PM	File	1 KB
main.py	6/4/2026 12:54 AM	Python.File	3 KB
model.pkl	6/4/2026 12:19 AM	PKL File	382 KB
requirements	6/4/2026 12:58 AM	Text Document	1 KB
train.py	6/4/2026 12:18 AM	Python.File	2 KB

The final directory containing only the localized tracking data, execution scripts (main.py, train.py), environment (requirements.txt), the model binary (model.pkl), and configuration files (Dockerfile, deploy.ps1).

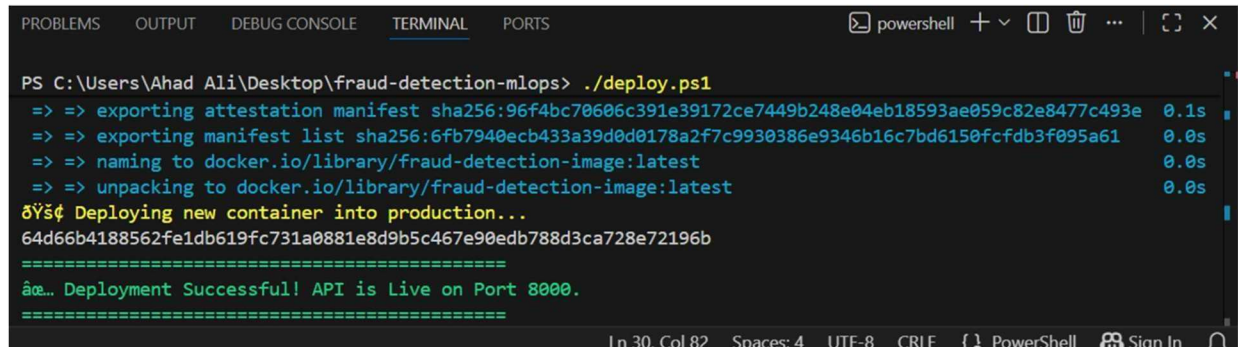
Containerization Environment (Docker Engine Status):



To achieve platform independence and isolated execution, Docker Desktop was initialized locally.

Automated CI/CD Deployment :

PowerShell Automated Pipeline Execution:

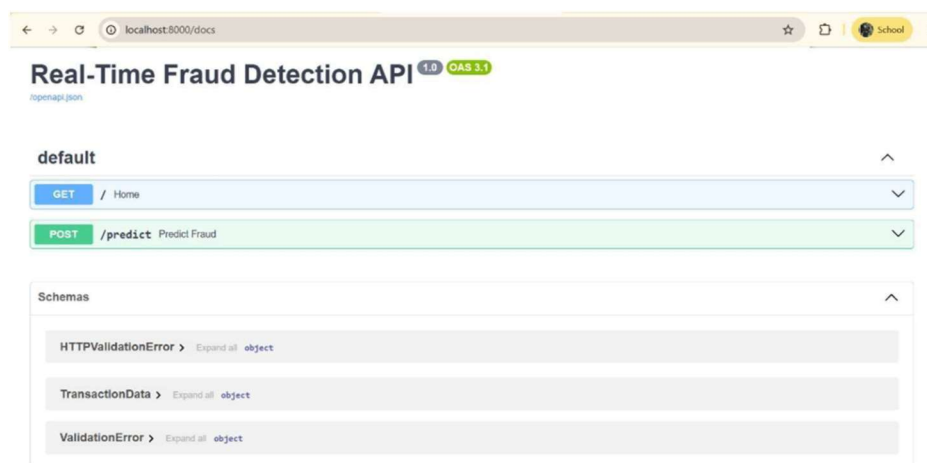


```
PS C:\Users\Ahad Ali\Desktop\fraud-detection-mlops> ./deploy.ps1
=> => exporting attestation manifest sha256:96f4bc70606c391e39172ce7449b248e04eb18593ae059c82e8477c493e 0.1s
=> => exporting manifest list sha256:6fb7940ecb433a39d0d0178a2f7c9930386e9346b16c7bd6150fcfdb3f095a61 0.0s
=> => naming to docker.io/library/fraud-detection-image:latest 0.0s
=> => unpacking to docker.io/library/fraud-detection-image:latest 0.0s
ðŸšŒ Deploying new container into production...
64d66b4188562fe1db619fc731a0881e8d9b5c467e90edb788d3ca728e72196b
=====
âœ… Deployment Successful! API is Live on Port 8000.
=====
```

A custom CI/CD automation script (deploy.ps1) was developed to manage the deployment lifecycle. The console logs confirm a flawless termination with the confirmation status: “Deployment Successful! API is Live on Port 8000”.

Production API Serving & Validation (FastAPI) :

FastAPI Swagger UI Interface:



Endpoint Schema & Request Payload Configuration:

GET / Home

POST /predict Predict Fraud

Parameters

No parameters

Request body required application/json

Edit Value Schema

```
{
  "Time": 0,
  "V1": 0,
  "V2": 0,
  "V3": 0,
  "V4": 0,
  "V5": 0,
  "V6": 0,
  "V7": 0,
  "V8": 0,
  "V9": 0,
  "V10": 0,
  "V11": 0,
  "V12": 0,
  "V13": 0,
  "V14": 0,
  "V15": 0,
  "V16": 0,
  "V17": 0,
  "V18": 0
}
```

Expanding the (Post / Predict) route reveals the structural JSON schema expected by the machine learning backend.

Automated Request Compilation (cURL Command):

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "Time": 0,
    "V1": 0,
    "V2": 0,
    "V3": 0,
    "V4": 0,
    "V5": 0,
    "V6": 0,
    "V7": 0,
    "V8": 0,
    "V9": 0,
    "V10": 0,
    "V11": 0,
    "V12": 0,
    "V13": 0,
    "V14": 0,
    "V15": 0,
    "V16": 0,
    "V17": 0,
    "V18": 0
  }'
```

Request URL

http://127.0.0.1:8000/predict

Server response

The API engine auto-compiles the transaction input payload parameters into an executive cURL request mapped directly against the loopback address destination ([<http://127.0.0.1:8000/predict>])(<http://127.0.0.1:8000/predict>)).

Model Testing and Real-Time Fraud Prediction Verification :



To ensure proper routing, model inference responses are thoroughly checked. The system evaluates incoming transactions against HTTP validation error criteria (422 Unprocessable Entity logs)

Live Output Inference Result (200 OK Response):



The framework successfully returned an HTTP 200 OK status code response in just a few milliseconds. The JSON response object contains the model's live decision output payload: (“prediction_code” : 0, “status” : “Legitimate”). This confirms the entire pipeline from training to automated container serving is fully functional.

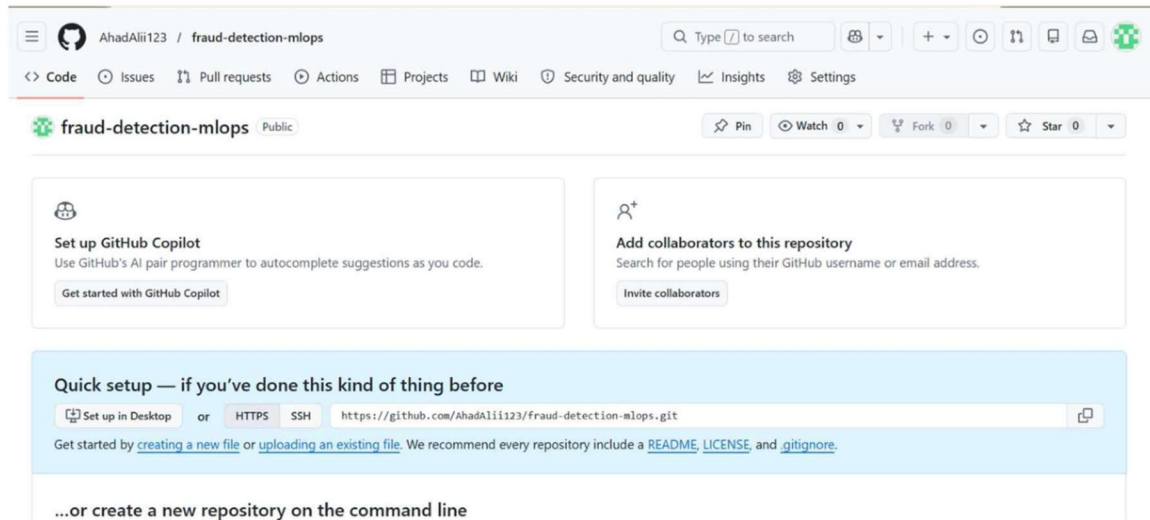
Version Control & Cloud Code Management (GitHub):

Local Git Repository Initialization & Configuration:

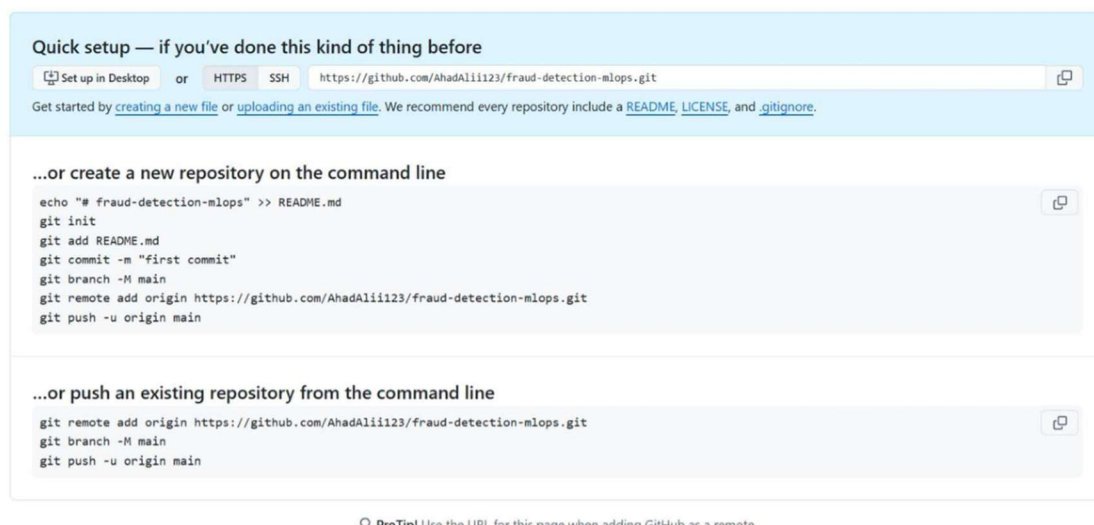
```
Ahad Ali@DESKTOP-334LMBR MINGW64 ~  
$ cd "/c/Users/Ahad Ali/Desktop/fraud-detection-mlops"  
  
Ahad Ali@DESKTOP-334LMBR MINGW64 ~/Desktop/fraud-detection-mlops (main)  
$ git init  
Reinitialized existing Git repository in C:/Users/Ahad Ali/Desktop/fraud-detection-mlops/.git/  
  
Ahad Ali@DESKTOP-334LMBR MINGW64 ~/Desktop/fraud-detection-mlops (main)  
$ git add .  
  
Ahad Ali@DESKTOP-334LMBR MINGW64 ~/Desktop/fraud-detection-mlops (main)  
$ git commit -m "Initial commit: End-to-End MLOps Fraud Detection Pipeline"  
On branch main  
nothing to commit, working tree clean
```

Using the Git Bash terminal, the workspace environment was configured to track all core execution scripts, configuration files, and tracking outputs.

Remote Cloud Mapping & Secure Deployment Setup:



A dedicated public repository was created on GitHub to host the project infrastructure.



The local setup was securely linked to this remote repository, ensuring a clean structure and setting up the foundation for potential CI/CD automation pipelines.

Automated Code Push Execution:

```
Ahad Ali@DESKTOP-334LMBR MINGW64 ~/Desktop/fraud-detection-mlops (main)
● $ git branch -M main

Ahad Ali@DESKTOP-334LMBR MINGW64 ~/Desktop/fraud-detection-mlops (main)
● $ git remote add origin https://github.com/AhadAlii123/fraud-detection-mlops.git

Ahad Ali@DESKTOP-334LMBR MINGW64 ~/Desktop/fraud-detection-mlops (main)
● $ git push -u origin main
Enumerating objects: 46, done.
Counting objects: 100% (46/46), done.
Delta compression using up to 14 threads
Compressing objects: 100% (34/34), done.
Writing objects: 100% (46/46), 105.39 KiB | 3.90 MiB/s, done.
Total 46 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), done.
To https://github.com/AhadAlii123/fraud-detection-mlops.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

The upload completed successfully without any errors or dependency issues.

Multi-Container Build Phase:

Pulling Base Images from Docker Hub :

```
Ahad Ali@DESKTOP-334LMBR MINGW64 ~/Desktop/fraud-detection-mlops (main)
$ docker compose up --build -d
time="2026-06-05T19:36:53+05:00" level=warning msg="C:\\Users\\Ahad Ali\\Desktop\\fraud-detection-mlops\\d
ocker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid pote
ntial confusion"
[+] up 23/23
✓ Image grafana/grafana:latest Pulled 76.6s
✓ Image prom/prometheus:latest Pulled 30.5s
#1 [internal] load local bake definitions
#1 reading from stdin 635B done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 667B done
#2 DONE 0.0s

#3 [internal] load metadata for docker.io/library/python:3.11-slim
#3 DONE 2.4s

#4 [internal] load .dockerignore
#4 transferring context: 2B done
#4 DONE 0.0s

#5 [internal] load build context
#5 transferring context: 2.00kB 0.1s done
#5 DONE 0.1s

#6 [1/6] FROM docker.io/library/python:3.11-slim@sha256:a3ab0b966bc4e91546a033e22093cb840908979487a9fc0e6e
38295747e49ac0
#6 resolve docker.io/library/python:3.11-slim@sha256:a3ab0b966bc4e91546a033e22093cb840908979487a9fc0e6e382,
```

Successful Service Stack Startup:

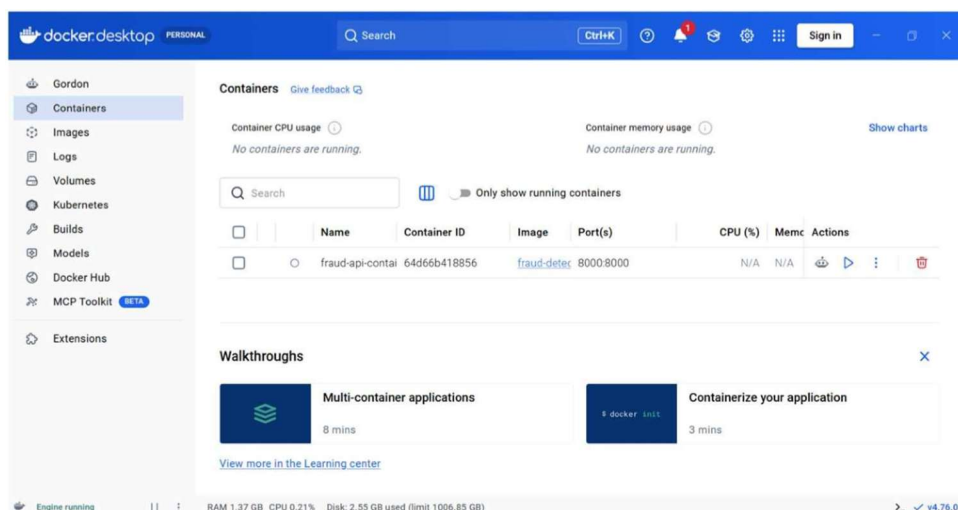
```
Ahad Ali@DESKTOP-334LMBR MINGW64 ~/Desktop/fraud-detection-mlops (main)
$ docker compose up --build -d
#11 CACHED

#12 exporting to image
#12 exporting layers done
#12 exporting manifest sha256:44d6d097da491df285f8d96210b80aad602e3bc3d8428c96e185d6aada67b099 0.0s done
#12 exporting config sha256:9d9153245a45dfb51f7d0afe8185c4d792afb0dacc146f0903b7c2732019ba2 done
#12 exporting attestation manifest sha256:ddbc644e0d9eb5d40531ab16f594c9ab3fc8bac3aa0052b635fb97cc18d26efa
0.0s done
#12 exporting manifest list sha256:2bf65f4897252c1a0a8105a649b3a12e956ca42f13834de68d6fbf94803ac34e
#12 exporting manifest list sha256:2bf65f4897252c1a0a8105a649b3a12e956ca42f13834de68d6fbf94803ac34e 0.0s d
one
#12 naming to docker.io/library/fraud-detection-mlops-fraud-api-container:latest done
#12 unpacking to docker.io/library/fraud-detection-mlops-fraud-api-container:latest 0.0s done
#12 DONE 0.2s

#13 resolving provenance for metadata file
#13 DONE 0.0s
[+] up 5/5
✓ Image fraud-detection-mlops-fraud-api-container Built 2.1s
✓ Network fraud-detection-mlops_monitoring_network Created 0.0s
✓ Container fraud-api-container Started 1.0s
✓ Container prometheus_container Started 1.2s
✓ Container grafana_container Started 1.5s

What's next:
  Filter, search, and stream logs from all your Compose services
  in one place with Docker Desktop's Logs view. docker-desktop://dashboard/logs?appId=fraud-detection-mlops
```

Docker Desktop Engine Initialization :



Verification of the Docker Desktop interface showing the global engine state as "Engine Running" before launching the stack.

FastAPI Live System Metrics Stream (Part A) :

```
← → ↻ ⓘ localhost:8000/metrics

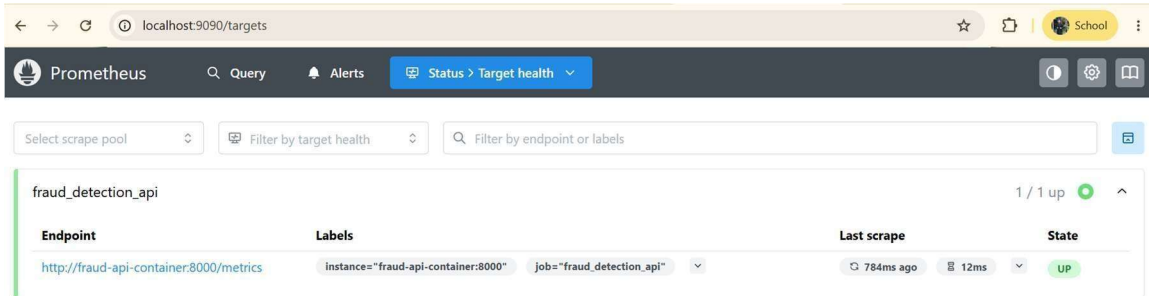
# HELP python_gc_objects_collected_total Objects collected during gc
# TYPE python_gc_objects_collected_total counter
python_gc_objects_collected_total{generation="0"} 772.0
python_gc_objects_collected_total{generation="1"} 458.0
python_gc_objects_collected_total{generation="2"} 20.0
# HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
# TYPE python_gc_objects_uncollectable_total counter
python_gc_objects_uncollectable_total{generation="0"} 0.0
python_gc_objects_uncollectable_total{generation="1"} 0.0
python_gc_objects_uncollectable_total{generation="2"} 0.0
# HELP python_gc_collections_total Number of times this generation was collected
# TYPE python_gc_collections_total counter
python_gc_collections_total{generation="0"} 370.0
python_gc_collections_total{generation="1"} 33.0
python_gc_collections_total{generation="2"} 3.0
# HELP python_info Python platform information
# TYPE python_info gauge
python_info{implementation="CPython",major="3",minor="11",patchlevel="15",version="3.11.15"} 1.0
# HELP process_virtual_memory_bytes Virtual memory size in bytes.
# TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes 1.5322112e+09
# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 1.77795072e+08
# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds 1.78067095067e+09
# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total 6.03
# HELP process_open_fds Number of open file descriptors.
# TYPE process_open_fds gauge
process_open_fds 11.0
# HELP process_max_fds Maximum number of open file descriptors.
# TYPE process_max_fds gauge
process_max_fds 1.048576e+06
# HELP http_requests_total Total number of requests by method, status and handler.
```

Validation of the instrumentation output on <http://localhost:8000/metrics>. This confirms the raw stream containing virtual memory bytes, process CPU cycles.

FastAPI HTTP Latency (Part B):

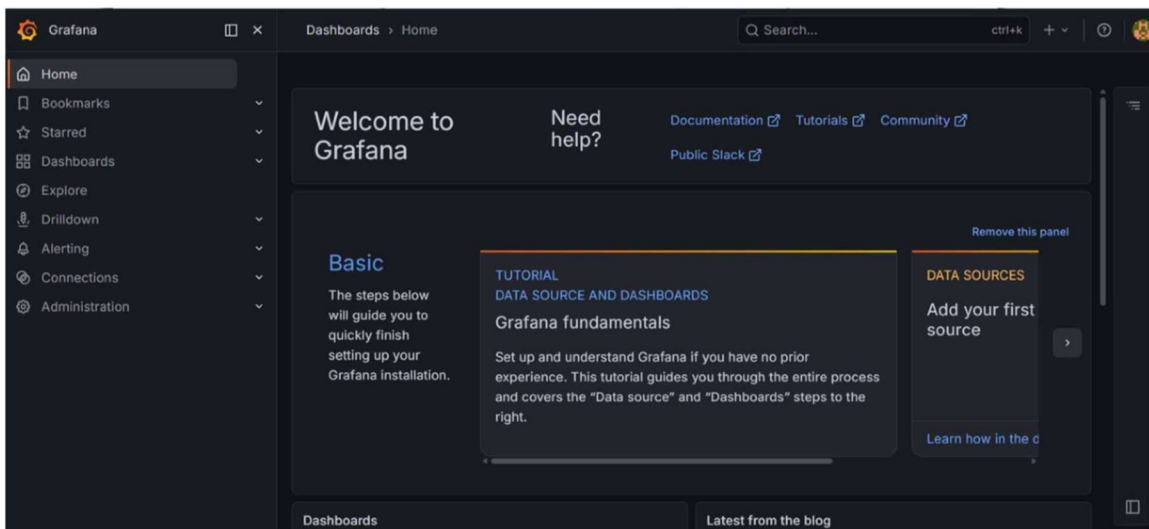
```
http_response_size_bytes_created{handler="/metrics"} 1.7806709607680595e+09
# HELP http_request_duration_highr_seconds Latency with many buckets but no API specific labels. Made for more accurate percentile calculations.
# TYPE http_request_duration_highr_seconds histogram
http_request_duration_highr_seconds_bucket{le="0.01"} 49.0
http_request_duration_highr_seconds_bucket{le="0.025"} 49.0
http_request_duration_highr_seconds_bucket{le="0.05"} 49.0
http_request_duration_highr_seconds_bucket{le="0.075"} 49.0
http_request_duration_highr_seconds_bucket{le="0.1"} 49.0
http_request_duration_highr_seconds_bucket{le="0.25"} 49.0
http_request_duration_highr_seconds_bucket{le="0.5"} 49.0
http_request_duration_highr_seconds_bucket{le="0.75"} 49.0
http_request_duration_highr_seconds_bucket{le="1.0"} 49.0
http_request_duration_highr_seconds_bucket{le="1.5"} 49.0
http_request_duration_highr_seconds_bucket{le="2.0"} 49.0
http_request_duration_highr_seconds_bucket{le="2.5"} 49.0
http_request_duration_highr_seconds_bucket{le="3.0"} 49.0
http_request_duration_highr_seconds_bucket{le="3.5"} 49.0
http_request_duration_highr_seconds_bucket{le="4.0"} 49.0
http_request_duration_highr_seconds_bucket{le="4.5"} 49.0
http_request_duration_highr_seconds_bucket{le="5.0"} 49.0
http_request_duration_highr_seconds_bucket{le="7.5"} 49.0
http_request_duration_highr_seconds_bucket{le="10.0"} 49.0
http_request_duration_highr_seconds_bucket{le="30.0"} 49.0
http_request_duration_highr_seconds_bucket{le="60.0"} 49.0
http_request_duration_highr_seconds_bucket{le="+Inf"} 49.0
http_request_duration_highr_seconds_count 49.0
http_request_duration_highr_seconds_sum 0.21206010699927447
# HELP http_request_duration_highr_seconds_created Latency with many buckets but no API specific labels. Made for more accurate percentile calculations.
# TYPE http_request_duration_highr_seconds_created gauge
http_request_duration_highr_seconds_created 1.7806709552989128e+09
# HELP http_request_duration_seconds Latency with only few buckets by handler. Made to be only used if aggregation by handler is important.
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{handler="/metrics",le="0.1",method="GET"} 49.0
http_request_duration_seconds_bucket{handler="/metrics",le="0.5",method="GET"} 49.0
http_request_duration_seconds_bucket{handler="/metrics",le="1.0",method="GET"} 49.0
http_request_duration_seconds_bucket{handler="/metrics",le="+Inf",method="GET"} 49.0
http_request_duration_seconds_count{handler="/metrics",method="GET"} 49.0
http_request_duration_seconds_sum{handler="/metrics",method="GET"} 0.21206010699927447
```

Prometheus Server Target Activation:



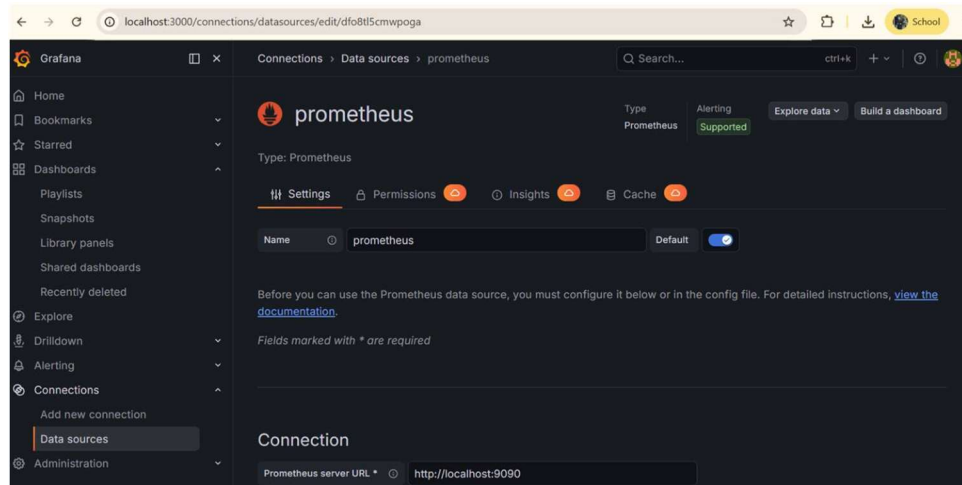
The internal target health state dashboard inside Prometheus running at `http://localhost:9090/targets`. It highlights the `fraud_detection_api` scraping endpoint configured correctly under the green UP operational state.

Grafana Visualization Management Portal :



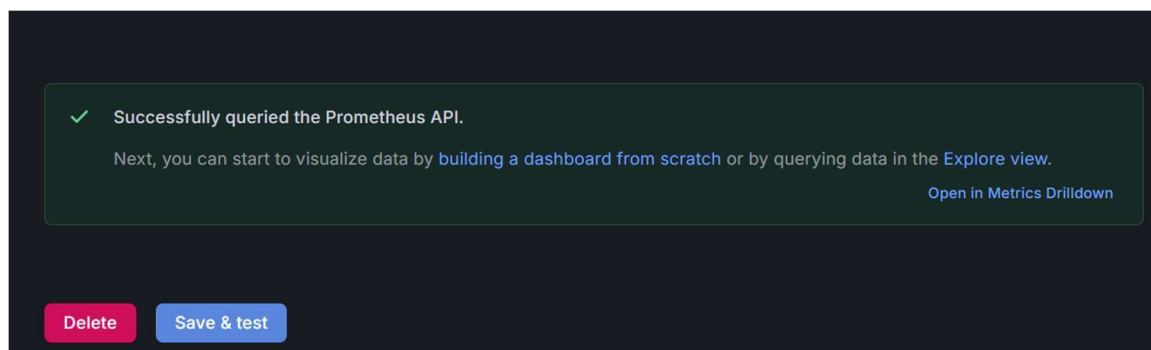
Prometheus Connection Configuration:

This setup configures the primary connection properties for the Prometheus data source by specifying the precise container network loopback address (`http://localhost:9000`).



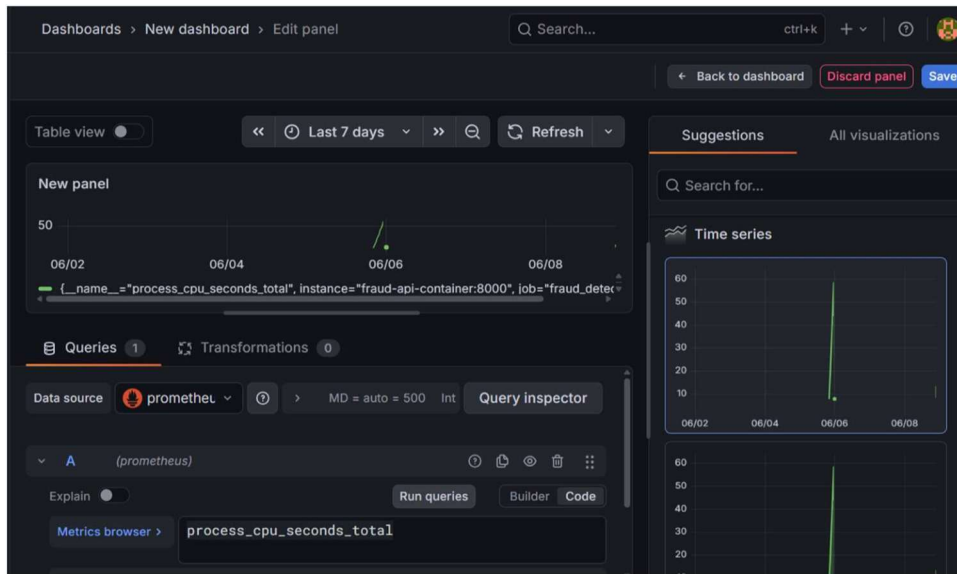
Prometheus API Endpoint Integration Testing:

A successful connectivity handshake verified that the Grafana instance can seamlessly query and fetch metrics data directly from the active Prometheus API engine.



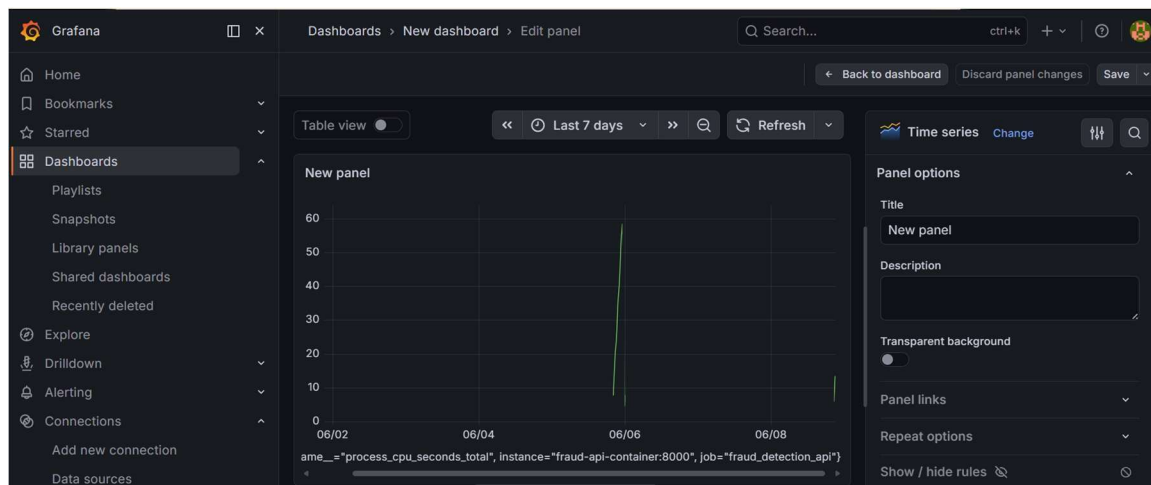
Metric Query Formulation and Real-Time Visualization:

This dashboard workspace captures the live graph generation process over a historical 7-day range by executing the target resource query `process_cpu_seconds_total`.



Completed Infrastructure Health and Performance Dashboard:

The finalized monitoring panel tracks container resource consumption trends, establishing full visibility over the underlying MLOps production cluster infrastructure.



MLflow Tracking Dashboard:

The MLflow tracking interface catalogs runtime experiments for the `fraud_detection_system`, systematically preserving lifecycle metrics, model artifacts, and parameter logs.

The screenshot shows the MLflow Tracking Dashboard. The left sidebar contains navigation links: Home, Experiments, Model registry, Assistant (Beta), Docs, and Settings. The main content area is titled 'Welcome to MLflow' and includes a 'Getting Started' dropdown. Below this is a 'New to MLflow?' section with a 'Start Demo' button. A horizontal carousel highlights features: Tracing, Evaluation, Prompts, and AI Gateway. At the bottom, the 'Recent Experiments' table lists three experiments: 'Fraud_Detection_System', 'Default', and an unnamed one.

Name	Time created	Last modified	Description	Tags
☐ Fraud_Detection_System	06/04/2026, 12:18:58 AM	06/04/2026, 12:18:58 AM	-	
☐ Default	06/04/2026, 12:18:58 AM	06/04/2026, 12:18:58 AM	-	
☐				

Verified Live Production Workspace on GitHub:

The production code is now live and fully updated on GitHub.

The screenshot displays the GitHub interface for the repository 'fraud-detection-mlops' owned by 'AhadAlii123'. The repository is public and has 4 commits. The file list shows the following files and their commit history:

File	Commit Message	Time
tracking_data	Initial commit: End-to-End MLOps Fraud Detection Pipeline	4 days ago
Dockerfile	Initial commit: End-to-End MLOps Fraud Detection Pipeline	4 days ago
Real_Time_Fraud_Detection_MLOps_Pipeline.pdf	Added final Real Time Fraud Detection MLOps report to rep...	3 days ago
deploy.ps1	Initial commit: End-to-End MLOps Fraud Detection Pipeline	4 days ago
docker-compose.yml	Added final Real Time Fraud Detection MLOps report to rep...	3 days ago
main.py	Added final Real Time Fraud Detection MLOps report to rep...	3 days ago
model.pkl	Initial commit: End-to-End MLOps Fraud Detection Pipeline	4 days ago